```
const express = require('express');
const cors = require('cors');
const fs = require('fs');
const path = require('path');
const mongoose = require('mongoose');

const app = express();
const PORT = process.env.PORT || 3000;
const MONGODB_URI = process.env.MONGODB_URI ||
'mongodb://127.0.0.1:27017/ghostbuildx';

app.use(cors());
app.use(express.json({ limit: '1mb' }));

mongoose.connect(MONGODB_URI)
  .then(() => console.log('MongoDB connected'))
  .catch((err) => console.error('MongoDB connection error:', err.message));

const projectSchema = new mongoose.Schema({
  projectName: { type: String, required: true },
  slug: { type: String, required: true, unique: true },
  description: { type: String, default: 'AI-generated starter project.' },
  stack: { type: String, default: 'Node.js + Express' },
  constraints: { type: [String], default: [] },
  features: { type: [String], default: [] },
  repoTree: { type: String, required: true },
  files: { type: Object, required: true }
}, { timestamps: true });

const Project = mongoose.model('Project', projectSchema);

function slugify(text = '') {
  return text
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, '-')
    .replace(/^-+|-+$/g, '')
    .slice(0, 60) || 'generated-project';
}

function buildProject(payload) {
  const name = payload.projectName || 'GhostBuild Project';
  const description = payload.description || 'AI-generated starter project.';
  const stack = payload.stack || 'Node.js + Express';
  const constraints = Array.isArray(payload.constraints) ? payload.constraints : [];
  const features = Array.isArray(payload.features) ? payload.features : [];
  const slug = slugify(name);

  const repoTree = [
```

```
    `${slug}/`,
    `├── README.md`,
    `├── AGENTS.md`,
    `├── package.json`,
    `├── src/`,
    `│   ├── app.js`,
    `│   ├── routes/`,
    `│   │   └── index.js`,
    `│   ├── services/`,
    `│   │   └── core.service.js`,
    `│   └── tests/`,
    `│       └── app.test.js`,
    `└── docs/`,
    `    └── architecture.md`
  ].join('\n');

  const readme = `# ${name}\n\n## Description\n${description}\n\n## Stack\n${stack}\n\n## Constraints\n${constraints.length ? constraints.map(c => `- ${c}`).join('\n') : '- None provided'}\n\n## Features\n${features.length ? features.map(f => `- ${f}`).join('\n') : '- Starter scaffold generation'}\n\n## Run\n1. npm install\n2. npm start\n\n## Test\n- npm test\n`;

  const agents = `# AGENTS.md\n\n## Project Goal\nBuild and extend ${name}.\n\n## Rules\n- Keep code simple and modular.\n- Do not rename API routes unless requested.\n- Run tests before finalizing changes.\n- Update README when adding major features.\n\n## Commands\n- Install: npm install\n- Start: npm start\n- Test: npm test\n\n## Important Notes\n- Respect project constraints: ${constraints.length ? constraints.join(', ') : 'none provided'}.\n- Preferred stack: ${stack}.\n`;

  return {
    projectName: name,
    slug,
    description,
    summary: `Generated starter package for ${name}`,
    stack,
    constraints,
    features,
    repoTree,
    files: {
      'README.md': readme,
      'AGENTS.md': agents,
      'docs/architecture.md': `# Architecture\n\nThis project uses ${stack}.\n\nCore features:\n${features.length ? features.map(f => `- ${f}`).join('\n') : '- Basic scaffold'}\n`,
      'package.json': JSON.stringify({
        name: slug,
        version: '1.0.0',
        private: true,
        main: 'src/app.js',
        scripts: {
```

```
        start: 'node src/app.js',
        test: 'echo "Add tests here"'
      }
    }, null, 2)
  }
 };
}

app.get('/', (req, res) => {
 res.json({
  ok: true,
  message: 'GhostBuild X backend is running',
  endpoints: {
   health: 'GET /health',
   generate: 'POST /api/generate-project',
   save: 'POST /api/projects',
   list: 'GET /api/projects',
   getOne: 'GET /api/projects/:id',
   deleteOne: 'DELETE /api/projects/:id'
  }
 });
});

app.get('/health', (req, res) => {
 res.json({
  status: 'healthy',
  uptime: process.uptime(),
  mongoReadyState: mongoose.connection.readyState
 });
});

app.get('/api/sample-project', (req, res) => {
 const sample = buildProject({
  projectName: 'Offline Event App',
  description: 'A starter repo for a low-data college event app.',
  stack: 'Node.js + Express + React',
  constraints: ['offline-first', 'low-end Android support'],
  features: ['event listing', 'local cache', 'simple admin panel']
 });
 res.json(sample);
});

app.post('/api/generate-project', (req, res) => {
 try {
  const result = buildProject(req.body || {});
  res.status(201).json(result);
 } catch (error) {
  res.status(500).json({ error: 'Failed to generate project', details: error.message });
```

```
  }
});

app.post('/api/projects', async (req, res) => {
  try {
    const generated = buildProject(req.body || {});
    const existing = await Project.findOne({ slug: generated.slug });

    if (existing) {
      existing.projectName = generated.projectName;
      existing.description = generated.description;
      existing.stack = generated.stack;
      existing.constraints = generated.constraints;
      existing.features = generated.features;
      existing.repoTree = generated.repoTree;
      existing.files = generated.files;
      await existing.save();
      return res.json({ message: 'Project updated in MongoDB', project: existing });
    }

    const project = await Project.create(generated);
    res.status(201).json({ message: 'Project saved to MongoDB', project });
  } catch (error) {
    res.status(500).json({ error: 'Failed to save project', details: error.message });
  }
});

app.get('/api/projects', async (req, res) => {
  try {
    const projects = await Project.find().sort({ createdAt: -1 });
    res.json(projects);
  } catch (error) {
    res.status(500).json({ error: 'Failed to fetch projects', details: error.message });
  }
});

app.get('/api/projects/:id', async (req, res) => {
  try {
    const project = await Project.findById(req.params.id);
    if (!project) {
      return res.status(404).json({ error: 'Project not found' });
    }
    res.json(project);
  } catch (error) {
    res.status(500).json({ error: 'Failed to fetch project', details: error.message });
  }
});
```

```
app.delete('/api/projects/:id', async (req, res) => {
  try {
    const deleted = await Project.findByIdAndDelete(req.params.id);
    if (!deleted) {
      return res.status(404).json({ error: 'Project not found' });
    }
    res.json({ message: 'Project deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: 'Failed to delete project', details: error.message });
  }
});

app.post('/api/export-project', async (req, res) => {
  try {
    let project;

    if (req.body.projectId) {
      project = await Project.findById(req.body.projectId);
      if (!project) {
        return res.status(404).json({ error: 'Project not found in MongoDB' });
      }
    } else {
      project = buildProject(req.body || {});
    }

    const exportDir = path.join(__dirname, 'exports', project.slug);
    fs.mkdirSync(exportDir, { recursive: true });

    Object.entries(project.files).forEach(([fileName, content]) => {
      const target = path.join(exportDir, fileName);
      fs.mkdirSync(path.dirname(target), { recursive: true });
      fs.writeFileSync(target, content, 'utf8');
    });

    res.json({
      message: 'Project exported locally',
      exportPath: exportDir,
      project
    });
  } catch (error) {
    res.status(500).json({ error: 'Export failed', details: error.message });
  }
});

app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});
```

```
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```